

Data Structures

COURSE CODE: CS-201

TOPIC: ARRAYS

COURSE INSTRUCTOR: RIDA ZAHRA



Goals for today

To study:

- Introduction to Arrays
- Types of Array
- Linear Array
- Length of Array

CLO Covered

CLO1: **Explain** the fundamentals of data structure as well as algorithms that operate on them

CLO2: **Apply** and design various data structures methods for specified applications.

Introduction to Array

An array is a collection of items of the same variable type that are stored at contiguous memory locations.

It is one of the most popular and simple data structures used in programming.

Arrays

What is an array?

- A linear structure approach (sequential memory location).
- A useful way of collecting the similar type of data items.

Example:

In case of Employee Record:

- Declare 50 different variables to store the age of employees.
- Assign values to each variable.

It is not easy to handle these variables in the program

The concept of array is useful in this situation

Note: Arrays can also be termed as Linear Arrays

Basic terminologies of Array

Array Element: Elements are items stored in an array.

Array Index: Elements are accessed by their indexes. Indexes in most of the programming languages start from 0.

Linear Arrays

A linear array is a list of a finite number of **n** homogeneous data elements (that is data elements of the same type) such that

- The elements of the arrays are referenced respectively by an index set consisting of **n** consecutive numbers
- The elements of the arrays are stored respectively in **successive memory locations**

Memory representation of Array

In an array, all the elements or their references are stored in contiguous memory locations. This allows for efficient access and manipulation of elements

```
int arr[] = {11, 9, 17, 89, 1, 90, 19, 5, 3, 23, 43, 99}
```



In C++, arrays are stored in contiguous memory, meaning that each element is placed right next to the previous one in memory.

Array in C++

Declaration of Array

Arrays can be declared in various ways in different languages.

// This array will store integer type element

```
int arr[5];
```

// This array will store char type element

```
char arr[10];
```

// This array will store float type element

```
float arr[20];
```

Initialization of Array

Arrays can be initialized in different ways in different languages.

```
int arr[] = { 1, 2, 3, 4, 5 };
```

```
char arr[5] = { 'a', 'b', 'c', 'd', 'e' };
```

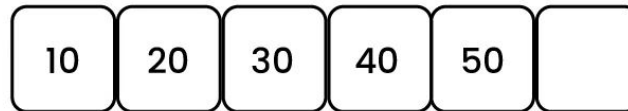
```
float arr[10] = { 1.4, 2.0, 24, 5.0, 0.0 };
```

Why do we Need Arrays?

Assume there is a class of five students and if we have to keep records of their marks in examination then, we can do this by declaring five variables individual and keeping track of records but what if the number of students becomes very large, it would be challenging to manipulate and maintain the data. So we use an array of student:

```
int v1 = 10;  
int v2 = 20;  
int v3 = 30;  
int v4 = 40;  
int v5 = 50;  
.  
.  
.
```

Multiple variables
to store each value



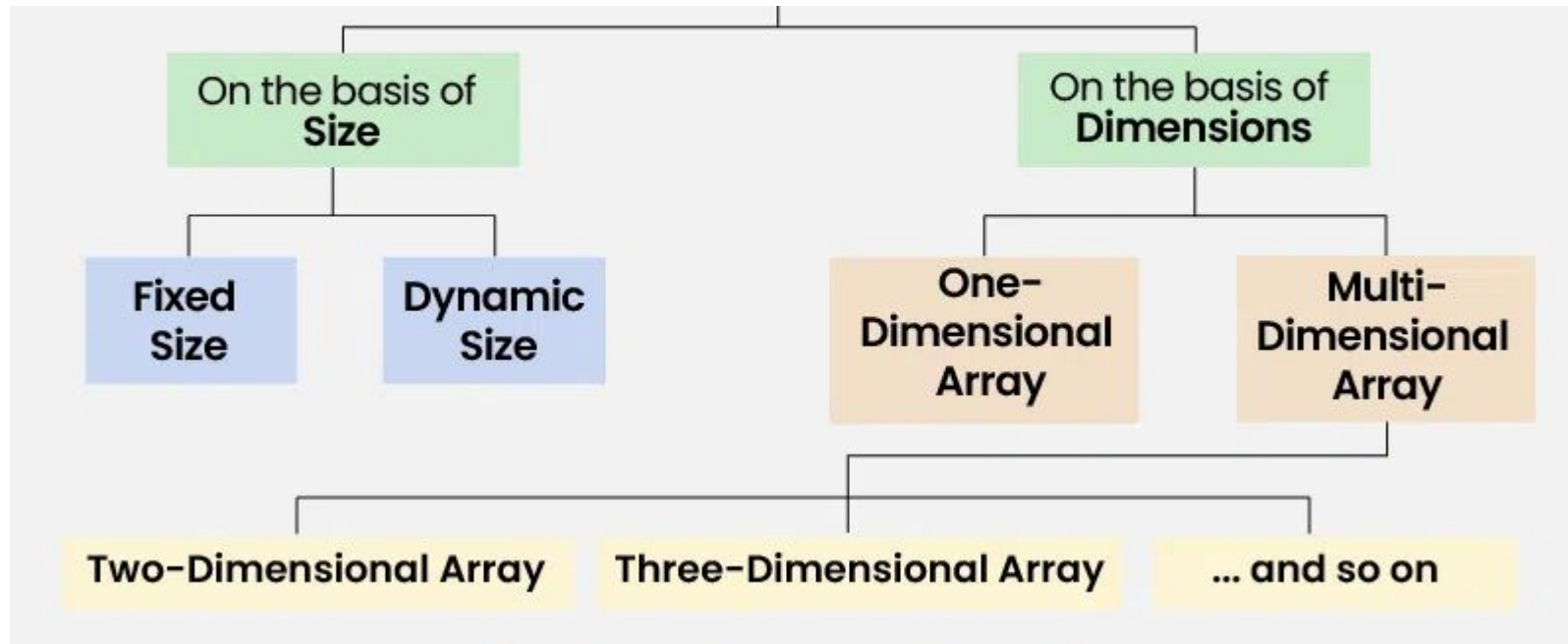
Single Array to store all values

Types of Arrays

Arrays can be classified in two ways:

On the basis of Size

On the basis of Dimensions



Types of Arrays on the basis of Size

1. Fixed Sized Arrays

- We cannot alter or update the size of this array. Here only a fixed size (i.e. the size that is mentioned in square brackets []) of memory will be allocated for storage.
- In case, we don't know the size of the array then if we declare a larger size and store a lesser number of elements, it will result in a wastage of memory. And if we declare a lesser size than the number of elements then we won't get enough memory to store all the elements.

Examples

// Here the memory is allocated at compile time.

```
int arr1[5];
```

// Another way (creation and initialization both)

```
int arr2[5] = {1, 2, 3, 4, 5};
```

2. Dynamic Sized Arrays

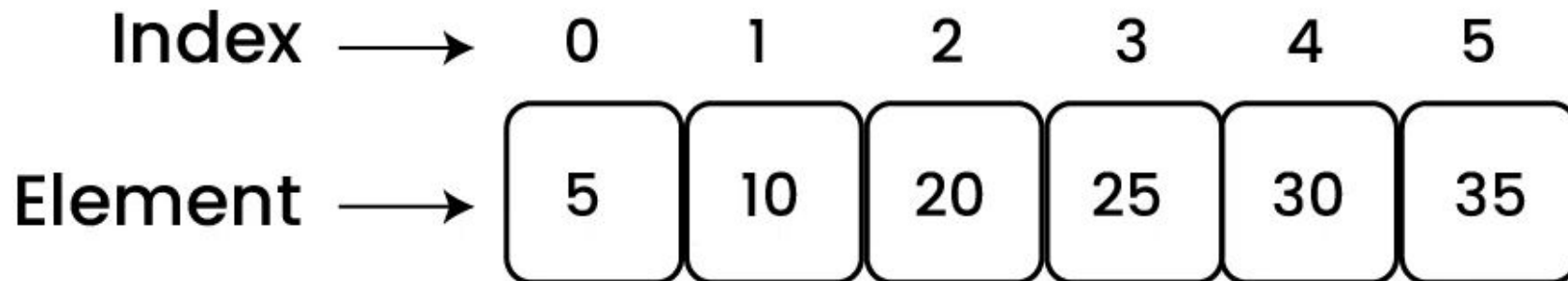
The size of the array changes as per user requirements during execution of code so the coders do not have to worry about sizes. They can add and removed the elements as per the need. The memory is mostly dynamically allocated and de-allocated in these arrays.

The size of the array changes as per user requirements during execution of code so the coders do not have to worry about sizes. They can add and removed the elements as per the need. The memory is mostly dynamically allocated and de-allocated in these arrays.

Types of Arrays on the basis of Dimensions

1. One-dimensional Array(1-D Array): You can imagine a 1d array as a row, where elements are stored one after another.

One-Dimensional Array (1-D Array)

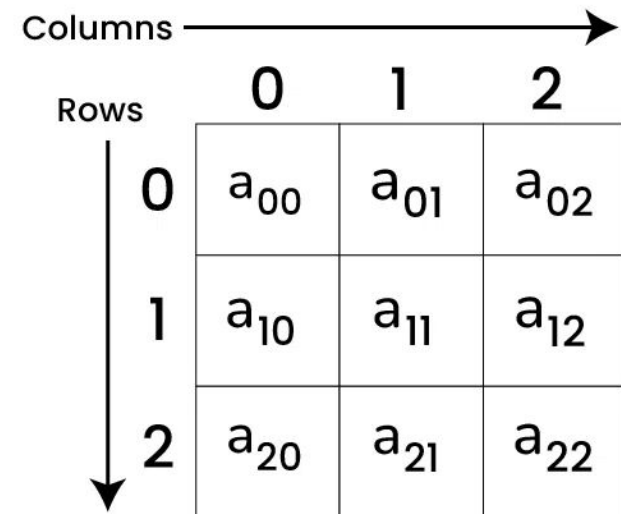


2. Multi-dimensional Array:

A multi-dimensional array is an array with more than one dimension. We can use multidimensional array to store complex data in the form of tables, etc. We can have 2-D arrays, 3-D arrays, 4-D arrays and so on.

Two-Dimensional Array(2-D Array or Matrix): 2-D Multidimensional arrays can be considered as an array of arrays or as a matrix consisting of rows and columns.

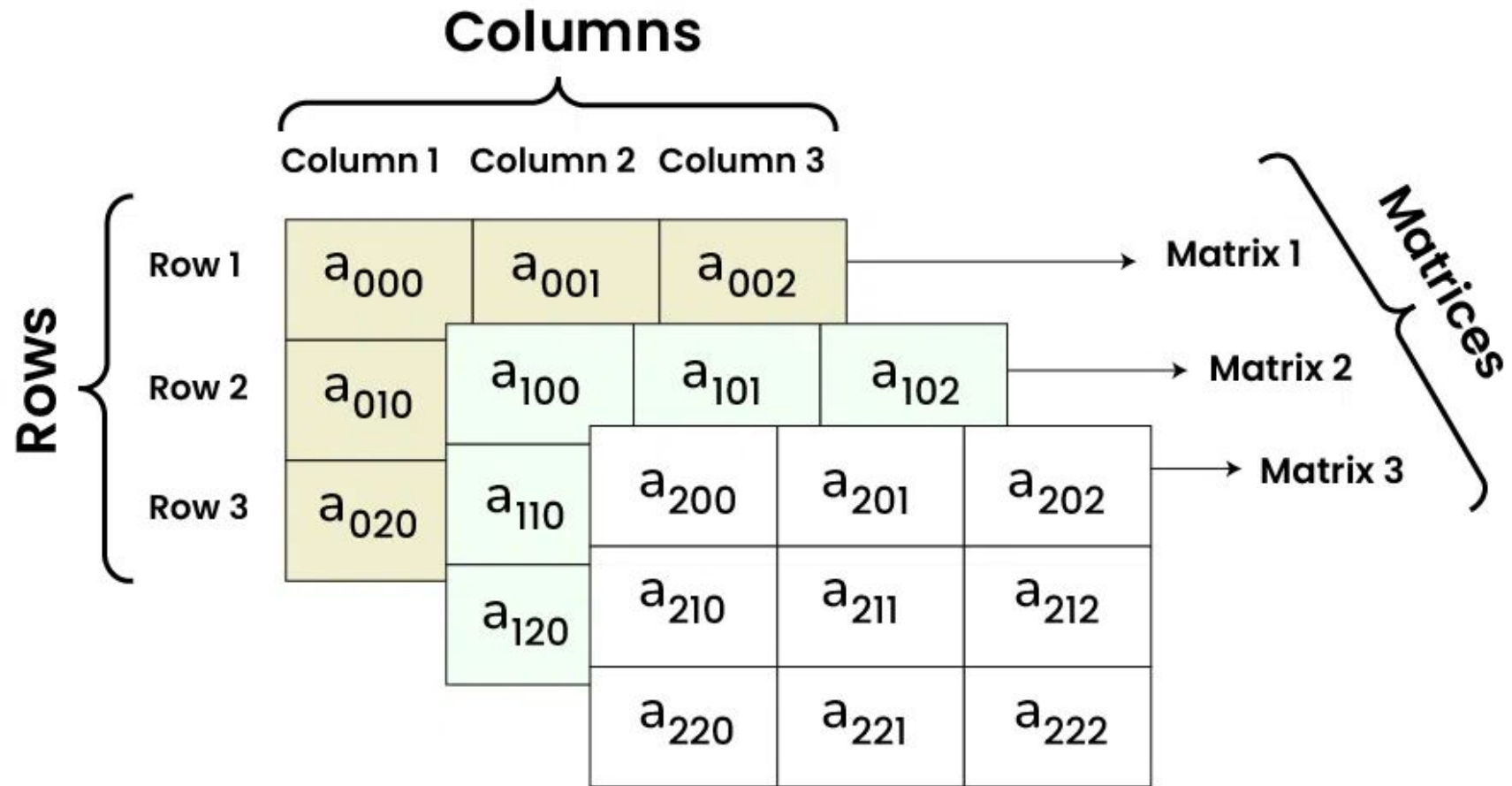
Two-Dimensional Array (2-D Array or Matrix)



The diagram illustrates a 2D array matrix. It features a grid of 3 rows and 3 columns. The columns are labeled 0, 1, and 2 at the top, with a horizontal arrow pointing to the right labeled 'Columns'. The rows are labeled 0, 1, and 2 on the left, with a vertical arrow pointing downwards labeled 'Rows'. Each cell in the grid contains an element labeled a_{ij} , where i is the row index and j is the column index.

	0	1	2
0	a_{00}	a_{01}	a_{02}
1	a_{10}	a_{11}	a_{12}
2	a_{20}	a_{21}	a_{22}

Three-Dimensional Array(3-D Array): A 3-D Multidimensional array contains three dimensions, so it can be considered an array of two-dimensional arrays.



What will be the output?

```
int main() {  
    int arr[5] = {2, 4, 8, 12, 16};  
    cout << arr[2] << " ";  
    cout << arr[4] << " ";  
    cout << arr[0] << " ";  
  
    return 0;  
}
```

Linear Arrays

- The number **n** of elements is called the **length or size** of the array.
- The index set consists of the integer **1, 2, ... n / 0,1,...n-1**
- **Length** or the number of data elements of the array can be obtained from the index set by:

$$\square \quad \text{Length} = \text{UB} - \text{LB} + 1$$

where **UB** is the largest index called the **upper bound** and **LB** is the smallest index called the **lower bound** of the arrays

Linear Arrays

Element of an array **A** may be denoted by

- Subscript notation **$A_1, A_2, \dots, A_n$**
- Parenthesis notation **$A(1), A(2), \dots, A(n)$**
- Bracket notation **$A[1], A[2], \dots, A[n]$**

Usually used the subscription notation or the bracket notation.

The number **K** in $A[K]$ is called subscript or an index and $A[K]$ is called a **subscripted variable**

Linear Array

Array DATA be a 6-element linear array of integers such that
DATA[0]=247, DATA[1]=56, DATA[2]=429, DATA[3]=135,
DATA[4]=87, DATA[5]=156..

247
56
429
135
87
156

247	56	429	135	87	156
-----	----	-----	-----	----	-----

Example

An automobile company uses an array `AUTO` to record the number of auto-mobiles sold each year from 1932 through 1984. Rather than beginning the index set with 1, it is more useful to begin the index set with 1932 so that $AUTO[K]$ = number of automobiles sold in the year K . Find the total number of elements in the array.

$LB = 1932$

$UB = 1984$

$Length = UB - LB + 1 = 1984 - 1932 + 1 = 53$

That is, `AUTO` contains 53 elements, and its index set contains of all integers from 1932 through 1984.

Representation of Linear Array in Memory

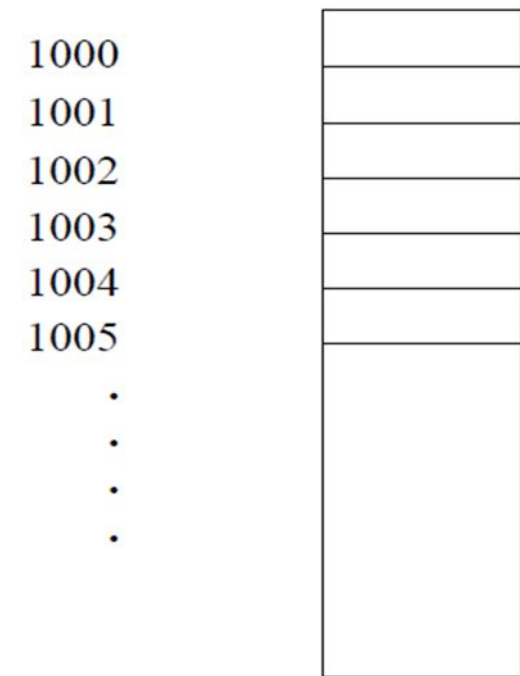
Let **LA** be a linear array in the memory of the computer

$\text{LOC}(\text{LA}[K])$ = address of the element $\text{LA}[K]$ of the array **LA**

The element of **LA** are stored in the successive memory cells

Computer does not need to keep track of the address of every element of **LA**, but need to track only the address of the first element of the array denoted by **Base(LA)** called the **base address** of LA

$\text{LOC}(\text{LA}[K]) = \text{Base}(\text{LA}) + w(K - \text{lower bound})$ where **w** is the number of words per memory cell of the array LA [**w** is a size of the **data type**]



Computer Memory

Representation of Linear Array in Memory

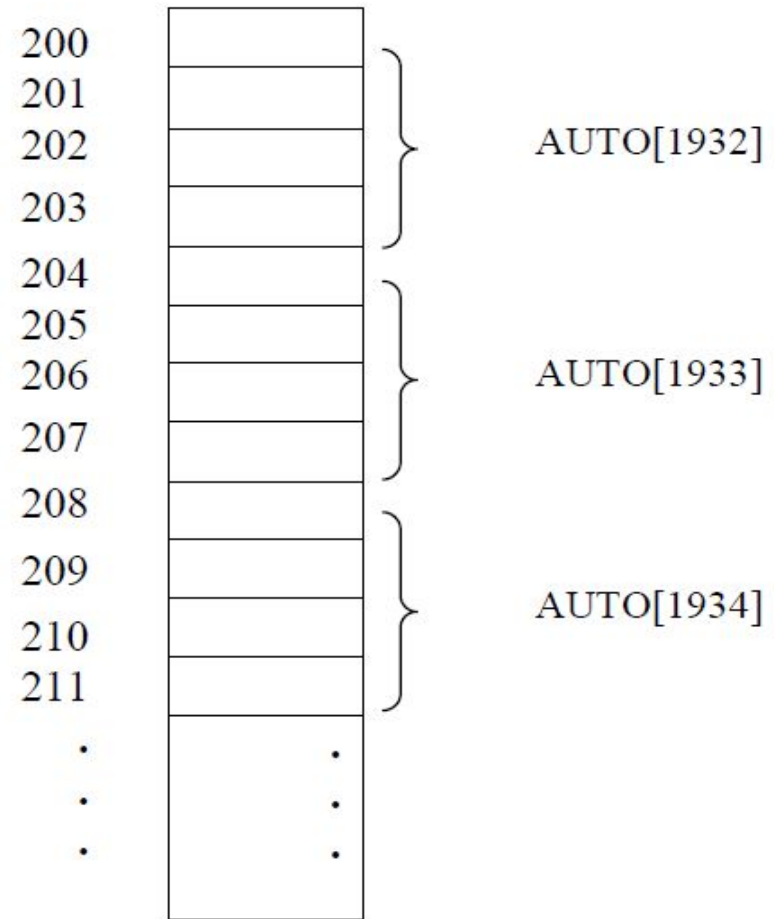
Consider the example array AUTO, which records the number of automobiles sold each year from 1932 through 1984. Array AUTO appears in memory is pictured in Fig. 01 (next slide) . Assume, $\text{Base}(\text{AUTO}) = 200$ and $w = 4$ words per memory cell for AUTO.

Then the base addresses of following arrays are,

$\text{LOC}(\text{AUTO}[1932]) = 200$, $\text{LOC}(\text{AUTO}[1933]) = 204$, $\text{LOC}(\text{AUTO}[1934]) = 208$,

Representation of Linear Array in Memory

- Let we find the address of the array element for the year $K = 1965$. It can be obtained by using Equation:
- $$\begin{aligned} \text{LOC}(\text{AUTO}[1965]) &= \\ &\text{Base}(\text{AUTO}) + w(1965 \\ &- \text{lower bound}) = 200 + \\ &4(1965 - 1932) = 332 \end{aligned}$$



Example

Find the address for LA[6]
Each element of the array occupy
1 byte

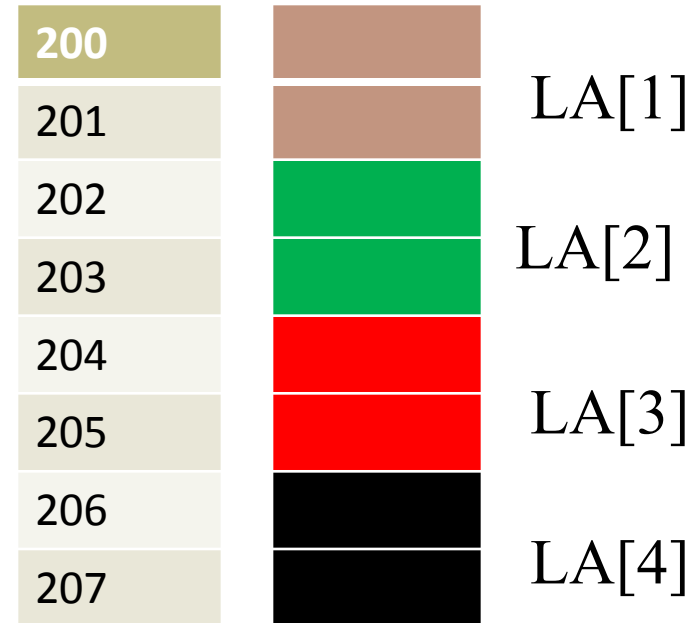
200		LA[1]
201		LA[2]
202		LA[3]
203		LA[4]
204		LA[5]
205		LA[6]
206		LA[7]
207		LA[8]

$$\text{LOC}(\text{LA}[\text{K}]) = \text{Base}(\text{LA}) + w(\text{K} - \text{lower bound}) \quad \vdots$$

$$\text{LOC}(\text{LA}[6]) = 200 + 1(6 - 1) = 205$$

Example 2

Find the address for LA[16]
Each element of the array occupy
2 byte



$$\text{LOC}(\text{LA}[\text{K}]) = \text{Base}(\text{LA}) + w(\text{K} - \text{lower bound})$$

$$\text{LOC}(\text{LA}[16]) = 200 + 2(16 - 1) = 230$$

Task

Given the base address of an array **A[1300.....1900]** as **1020** and the size of each element is 2 bytes in the memory, find the address of **A[1700]**?

task

Given the base address of an array **B[500...1000]** as **2040**, and each element occupies **4 bytes** in memory, find the address of **B[750]**.

Data Structures

COURSE CODE: CS-201

TOPIC: ARRAYS

COURSE INSTRUCTOR: RIDA ZAHRA

A solid orange horizontal bar spanning the entire width of the slide at the bottom.

Today's Goal

- Operations on Array

CLO Covered in this Lecture

CLO2: **Apply** and design various data structures methods for specified applications.

Operations on Linear Array

We can perform various operations on a linear array:

- **Traversing**- processing each element of the array list.
- **Inserting**- adding new elements in the array list.
- **Deleting**- removing an element from the array list.
- **Sorting**- arranging the elements of the list in some sorting order.
- **Merging**- combining the elements of two array lists in a single array list.

Overflow & Underflow

Overflow: Attempt to insert an element into an array exceeding its size limit.

Underflow: Attempt to delete or traverse an element from an empty array.

Traversal in Array

Traversal in an array refers to the process of accessing each element in the array sequentially, typically to perform a specific operation, such as searching, sorting, or modifying the elements.

Traversing an array is essential for a wide range of tasks in programming, and the most common methods of traversal include iterating through the array using loops like for, while, or foreach. Efficient traversal plays a critical role in algorithms that require scanning or manipulating data.

Traversing Linear Arrays

Traversing is accessing and processing (visiting) each element of the data structure exactly ones

Linear Array



Traversing Linear Arrays

Traversing is accessing and processing (visiting) each element of the data structure exactly ones

Linear Array



Traversing Linear Arrays

Traversing is accessing and processing (visiting) each element of the data structure exactly ones

Linear Array



Traversing Linear Arrays

Traversing is accessing and processing (visiting) each element of the data structure exactly ones

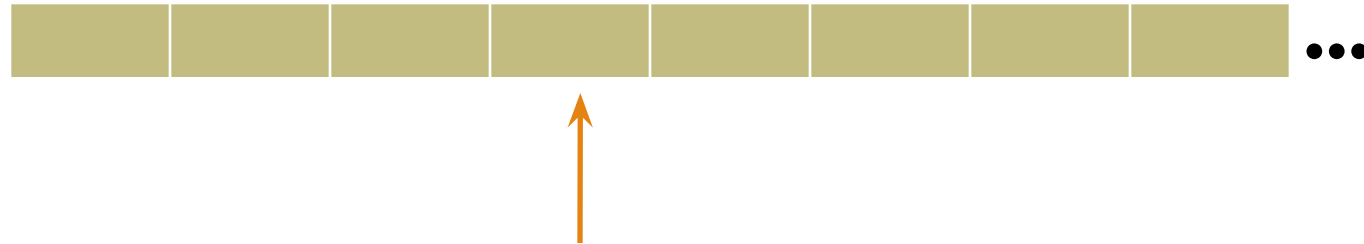
Linear Array



Traversing Linear Arrays

Traversing is accessing and processing (visiting) each element of the data structure exactly ones

Linear Array



Traversing Algorithm

We can process each element of an array with the help of an index set.

Consider a Linear Array(LA) list given with lower bound(LB) and upper bound(UB) that contains 'n' number of elements. We can traverse the list with the given algorithm.

1. [Initialize counter] Set $K := LB$.
2. Repeat Steps 3 and 4 while $K \leq UB$
3. [Visit element] Apply PROCESS to $LA[K]$
4. [Increase counter] Set $K := K + 1$
- [End of Step 2 loop]
5. Exit.

Searching in Linear Arrays

Search '4' in the following linear array:

10	3	5	9	4	8	1	2
----	---	---	---	---	---	---	---



Examples:

Input: arr[] = [10, 20, 30, 40, 50]

Output: "10 20 30 40 50 "

Explanation: Just traverse and print the numbers.

Input: arr[] = [7, 8, 9, 1, 2]

Output: "7 8 9 1 2 "

Explanation: Just traverse and print the numbers.

Input: arr[] = [100, 200, 300, 400, 500]

Output: "100 200 300 400 500 "

Explanation: Just traverse and print the numbers.

Applications of Array Traversal

Array traversal plays a key role in many operations where we need to access or modify the elements of an array. Two of the most common applications are **searching elements** and **modifying elements**.

1. Searching Elements

Traversal is often used to search for an element in an array. By visiting each element of the array sequentially, we can compare each element with the target value to determine if it exists.

Example: If you have an array of numbers, and you need to find if a specific number is present in the array, array traversal would involve checking each element one by one until a match is found or the entire array has been searched.

2. Modifying Elements

Array traversal is also used to modify the elements of the array. This could involve updating values, changing the order, or applying some mathematical operations to each element. For example, you can multiply every element of an array by a constant, or increase each value by 1.

Example: *If you want to increase each element in an array by 5, array traversal allows you to visit each element and modify it.*

```
int main() {  
    int arr[] = {10, 20, 30, 40, 50};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    // Modifying elements using traversal  
    (increasing each by 5)  
    for(int i = 0; i < n; i++) {  
        arr[i] += 5;  
    }  
  
    // Print modified array  
    cout << "Modified array: ";  
    for(int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
  
    return 0;  
}
```


Traversing Linear Arrays

Traversing is accessing and processing (visiting) each element of the data structure exactly ones

Linear Array



Types of Searching in Arrays

□ Linear Search

□ Binary Search (can only be applied on sorted array)

Linear / Sequential Search

Sequential search is also known as linear or serial search. It follows the following step to search a value in array.

- Visit the first element of array and compare its value with required value.
- If the value of array matches with the desired value, the search is complete.
- If the value of array does not match, move to next element and repeat same process up till to the length of array until the desired element is found.

Linear Search Algorithm

1. Start
2. Set $i = 0$
3. Repeat steps 4 & 5 while $i < UB$
4. IF ($item == LA[i]$) THEN GOTO Step 6
5. Set $i = i + 1$
6. Print 'record found at index i '
7. Print 'item not found'
8. Stop

Time Complexity

Running time for Linear Search is $O(n)$ as it depends on input size.

Binary Search

Binary search is a quicker method of searching for value in the array. But binary search is **applied only on sorted array**.

1. It locates the middle element of array and compare with desired number.
2. If they are equal, search is successful and the index of middle element is returned.
3. If they are not equal, it reduces the search to half of the array.
4. If the search number is less than the middle element, it searches the first half of array.

Otherwise it searches the second half of the array. The process continues until the required number is found or loop completes without successful search.

Binary Search Algorithm

if $UB == -1$, then print 'underflow' & return.

Repeat while $(LB \leq UB)$

{

mid = $(LB + UB) / 2$;

if $(LA[mid] == item)$

print 'item found'; return $LA[mid]$;

elseif $(item < LA[mid])$

UB = mid - 1;

else

LB = mid + 1;

}

print 'item not in list';

Time Complexity

Running time for Binary Search is $O(\log n)$ as it divides the array in two half if required element is not found

Questions?

